Project Report on Hospital Management App - V2

1. Student Details

Name: Anshul Shakya
Roll No.: 24f1002114
Email: 24f1002114@ds.study.iitm.ac.in

2. Project Details

Approach:

The application was developed using a decoupled architecture: a VueJS Single Page Application (SPA) for the frontend and a Flask backend serving RESTful APIs via Flask-RESTful. Persistence is handled by SQLite/SQLAlchemy. Role-based access control is implemented using Flask-Security and tokens. Performance is enhanced via Redis caching, and asynchronous tasks (scheduled reminders, reports, CSV export) are managed by Celery using Redis as the message broker.

3. AI/LLM Declaration

ChatGPT (GPT-5) was used only for minor code suggestions and documentation refinement, constituting approximately 10–15% of the work. All core development, debugging, and system integration were completed independently

4. Technologies and Frameworks Used

Technology Stack

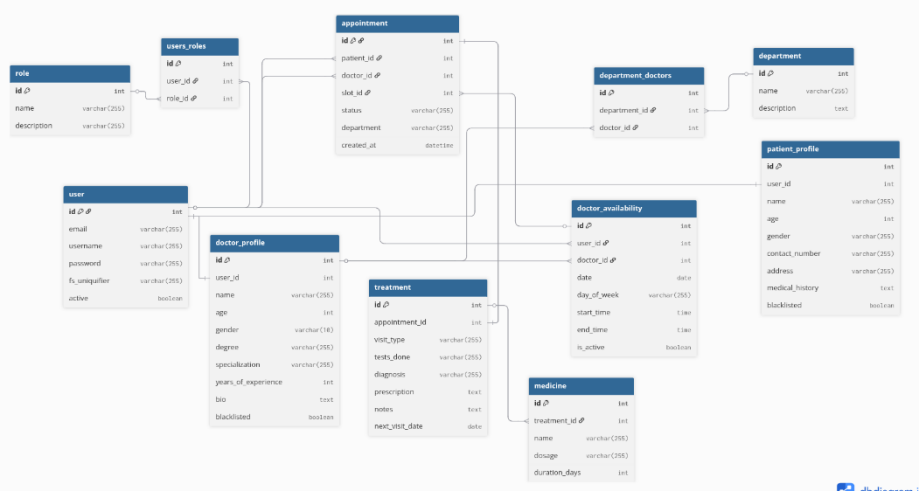
Technology	Purpose
Flask	Backend framework powering REST API and request routing
Vue.js	Dynamic SPA framework for responsive, interactive UI
Bootstrap	UI styling and mobile-first layout consistency
SQLite	Lightweight relational database for application persistence
SQLAlchemy / Flask-SQLAlchemy	ORM abstraction layer for simplified database operations
Redis / Flask-Caching	High-performance caching for frequently accessed data
Celery	Asynchronous task queue for alerts, reports, and exports
Flask-Security	RBAC enforcement for role-aware access control
Flask-RESTful	build REST APIs with resource-based routing and clean structure.
MailHog	email capture tool used for testing outbound notifications without hitting real inboxes.
Jinja2	Minimal templating for email html templates

5. Database Schema / ER Diagram

Database Tables

Table	Description
User	Centralized authentication and base profile for all roles
Role	Stores predefined roles (Admin, Doctor, Patient)
PatientProfile	Patient-specific demographic and medical details
DoctorProfile	Doctor-specific professional details
Department	Medical specializations/departments
DoctorAvailability	Doctor's scheduled available time slots
Appointment	Scheduled meeting records
Treatment	Medical records for completed appointments
Medicine	Prescribed drugs linked to treatments

ER - Diagram:



Database Relationships

Relationship	Type
User ↔ Role	Many-to-Many
User → PatientProfile	One-to-One
User → DoctorProfile	One-to-One
DoctorProfile ↔ Department	Many-to-Many
DoctorProfile → Availability	One-to-Many
Availability → Appointment	One-to-One
User → Appointment (patient/doctor)	One-to-Many
Appointment → Treatment	One-to-One
Treatment → Medicine	One-to-Many

6. API Resource Endpoints

Endpoint	Methods	Description
/api/login	POST	Authentication + token issuance
/api/doctors	GET, POST	Retrieve or onboard doctor profiles
/api/doctor/<id>	GET, PUT, DELETE	Full doctor profile lifecycle management
/api/patients	GET	Fetch patient profile inventory
/api/patient/<id>	GET, PUT, DELETE	View/update/retire patient record
/api/user/<id>/blacklist	POST	Activate/deactivate user access
/api/departments	GET, POST	Access or expand medical specialties
/api/department/<id>	GET, PUT, DELETE	Manage specific department entity
/api/appointments	GET, POST	Query or book appointments
/api/appointment/<id>	GET, PUT, DELETE	Reschedule/cancel/update appointments
/api/treatments	GET, POST	View history or log medical outcomes
/api/treatment/<id>	GET, PUT, DELETE	Maintain individual treatment record
/api/availabilities	GET, POST	Create or inspect availability slots
/api/doctor/availability/<id>	PUT, DELETE	Update or remove availability slot
/api/search/doctors	GET	High-velocity doctor discovery (cached)
/api/search/patients	GET	Rapid patient record lookup

YAML API Definition File: Included separately in the submission ZIP as **api.yaml**

7. Architecture Overview

Folder Structure:

```
Hospital_Management_System_V2_24f1002114/
├── app.py                # Application entry point
├── application/
|   ├── models.py         # SQLAlchemy database models
|   ├── resources/        # Flask-RESTful API resources
|   ├── tasks.py          # Celery background tasks
|   └── config.py         # Application configuration
├── static/               # Vue.js SPA (Vue Components), CSS files etc.
├── templates/            # Index.html (Jinja2 templates for mail)
├── instance/             # SQLite database file
├── api.yaml              # OpenAPI specification
├── requirements.txt      # Python dependencies
└── README.md
```

Technologies: Flask + SQLAlchemy + Celery + Redis
Entry Point: app.py bootstraps security, database, REST APIs, and async workers
Architecture Layers:

- **Data Layer**: SQLAlchemy models (User, Appointment, Treatment, Medicine, etc.)
- **API Layer**: Role-aware REST endpoints with Flask-RESTful
- **Frontend:** Vue.js SPA
- **Task Queue:** Celery workers with Redis broker for async operations

8. Implemented Features

Core Functionalities
Role-Based Access Control (RBAC)

- Token-based authentication via Flask-Security
- Strict role enforcement (Admin/Doctor/Patient) on all API endpoints
- Pre-seeded Admin user created on database initialization

Admin Operations

- Full CRUD for doctor profiles
- Blacklist/unblack list users to control access
- Department management
- Doctor search by specialization or name
- Patient search by ID or Name

Patient Operations

- Self-registration and profile management
- Doctor search by department and availability
- Appointment booking with conflict detection
- Cancel and reschedule appointments
- View treatment history

Doctor Operations

- Set availability for next 7 days
- View assigned appointments dashboard
- Create treatment records (diagnosis, prescriptions)

Conflict Prevention

- Logic ensures single booking per doctor/time slot
- Prevents double-booking at database level

Background Jobs (Celery + Redis)

Daily Reminders (Scheduled)

- Celery Beat job runs daily
- Sends appointment mail to patients

Monthly Activity Reports (Scheduled)

- Runs on 1st of each month
- Compiles doctor statistics (appointments, treatments)
- Generates HTML reports for email delivery

CSV Export (User-Triggered)

- Async Celery task for patient treatment history
- Generates downloadable CSV file

Performance & Caching

Redis Caching (Flask-Caching)

- Applied to high-frequency, low-change endpoints
- Cached resources: doctor search results, departments, registration data
- TTL: 5-10 minutes for optimal balance
- Reduces database load and improves response times

9. Video Presentation

Link : https://drive.google.com/file/d/1YXH7u_YSsdtpVcE6MkePMGOm_9Dxd5gy/view?usp=sharing